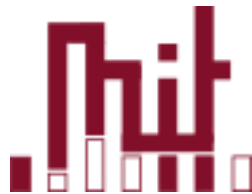


Beágyazott információs rendszerek

dr. Kovácsházy Tamás
8. labor forgatókönyv



Mesterséges Intelligencia és
Rendszertervezés Tanszék

GPIO használata

GPIO sebessége
Ütemezés vizsgálata GPIO-val

Ütemezés vizsgálata

- Hogyan vizsgálhatjuk az ütemezést?
- Hát pl. GPIO-val
 - Ha van egy olyan task-unk, ami folyamatosan változtatja egy GPIO értékét, akkor látjuk, hogy mikor fut...
 - Több GPIO PIN esetén akár több task-ot is tudunk vizsgálni
 - Az adott task adott GPIO PIN-t kezel
 - A GPIO kimenet frekvenciája a megoldás sebességét mutatja
 - Ez függ a HW-tól
 - A programozási nyelvtől és SW architektúrától
 - A GPIO elérés módjától
 - Persze ehhez kell egy logikai analizátor vagy oszcilloszkóp
 - De ilyen van... 😊
 - Meg az sem árt, ha beállítjuk a GPIO-t kimenetnek
 - `cat /sys/class/gpio/gpioX/direction`
 - `echo out > /sys/class/gpio/gpioX/direction`

Olcsó, de működőképes logikai analizátor

- Saleae klón és eredeti szoftver (vagy open source)
 - Max. 24 MHz mintavételi frekvencia (nekünk elég lesz most)
 - Az USB kábelekkel lesz probléma
 - Valahogy nem sikerül az USB csatlakozókat és kábeleket szabványosra legyártani
 - Nem közbeszerzésben kerültek az egyetemre ☹
 - Amit láttok, az itt sincs...
 - Pont, mit az EDU CAPE is...
- A Saleae Logic nagyon rendes, hogy nem tiltja le az SW-t (megtehetnék)



Az eredeti Saleae Logic műszer

- Csak hogy lássuk a különbséget...
- A jelenleg kaphatók specifikációja is sokkal jobb
 - Nem kellene azok a képességek most nekünk
 - 20+ beszerzése megterhelné a családi költségvetést...

Everything Included



- Saleae Logic 2
 - <https://home.mit.bme.hu/~khazy/bir2026/Logic-2.4.44-linux-x64.ApplImage>
 - Appimage (és annak minden szépsége)
 - `chmod 754 filename`
 - `./filename --no-sandbox`
 - udev rule-ok installálása
 - kilépés/belépés, vagy „`sudo service udev restart`”
 - eszköz kihúzása, bedugása
 - Erre töltődik be az új szabállyal az eszköz
 - szoftver újraindítása
 - utána szokott menni...
- Ha nem tetszik, akkor sigrok (open source alternativa)
 - `sudo apt install sigrok`
 - pulseview
 - udev rules...
 - Nekem most valami miatt instabil ezzel az analízátorral...

0. feladat

- Készítsünk bash scriptet, ami végtelen ciklusban írja 0 és 1-gyel a gpio66-ot
 - Ehhez kimenetnek kell beállítani a gpio66-ot
- Nézzük meg, milyen gyorsan tudja ezt megcsinálni

1. feladat

- A sysfs alapú GPIO elérést használjuk a GPIO66-on
 - `/sys/class/gpio/gpio66/...`
- Feladat:
 - Kimenetnek állítsuk be a GPIO-t (ha már nem tettük meg)
 - De egyébként be kéne állítani a „direction” file-ba „out”-t írva
 - File megnyitása írás joggal (`../value`, fp-t kapjuk meg)
 - Ha sikeres, végtelen ciklusban (`while(1)`):
 - „1”-t írunk bele (pl. `fprintf`)
 - ~~`sleep(1)`~~
 - „0”-t írunk bele (pl. `fprintf`)
 - ~~`sleep(1)`~~
- File-t az OS buffereli, a buffert üríteni kell...
 - `fflush(fd)`
- Mit látunk, mekkor a GPIO változási frekvenciája?
 - Írjuk fel egy doksiba...
- Mennyire folyamatos a változás?
 - Vannak-e benne pl. kisebb kimaradások?
 - Ha igen, milyen hosszúak, és miért vannak ott?
 - Ezeket is írjuk fel egy doksiba

2. feladat

- Írjunk (töltsük le) egy második programot, amivel ugyan így kezelhetjük a GPIO67-t
 - -6/-7 parancssori opcióval kiválasztva a gpio66-ot vagy a gpio77-et
- Futtassuk a két programot párhuzamosan (több SSH-t megnyitva pl.)
 - Mit látunk? Milyenek az időzítések, írjuk ezeket is fel?
 - Mennyi ideig fut az egyik, a másik, és amikor váltunk, mennyi ideig nem fut egyik sem (mi fut ilyenkor?)
 - A BBB-re telepítve van a stress-ng nevű terhelés generátor
 - Lépünk be egy 3. SSH-val ehhez
 - Csináljunk vele CPU terhelés (1-2-4 CPU HOG)
 - Csináljunk vele I/O (file) terhelést
 - Mit változtat meg ez?
 - Nézzük meg a chrt parancsot
 - Mi van ha az egyik, a másik, vagy mindkét gpio kezelő programot FIFO vagy RR ütemezővel ütemezzük a default ütemező helyett?
 - Hogyan változnak meg az időzítések?
 - Mit éreztünk a terminálon, ha valamelyik task RT ütemezéssel fut?
 - Miért nem életszerű az itt alkalmazott RT terhelés?

chrt használata már futó programon

- `chrt -m` : Egyes ütemezési osztályokra értelmezett prioritások
- `chrt -p PID` : az ütemezés lekérdezése
- `chrt -p -f priority PID` : fifo ütemezéssel priority prioritással
 - Gyakorlatban használható ütemezési osztályok
 - `-r/--rr priority`
 - `-f/--fifo priority`
 - `-o/--other 0` (priority optional, must be zero, is specified)
 - `-d/--deadline`, az ütemezés további paramétereit meg kell adni

A sysfs GPIO elérés nem túl gyors

- A chardev sem lenne sokkal gyorsabb...
- Miért? Mindkét esetben rendszerhívást kell végrehajtani
- Megoldás:
 - Turkáljunk direkt a memóriában...
 - 1. feladat: Megnézni a GPIO66 és GPIO67 regiszter címeit
 - A Reference Manualban kell turkálnunk
 - Megkeresni a GPIO2 címterületét a memory map-ben (183. oldal)
 - Azon belül a SET és CLEAR regisztereket (25.4.1 GPIO Registers, 4990 oldal)
 - 2. feladat írni (letölteni) egy olyan, programot, ami
 - Bemap-eli ezeket a fizikai címeket a mi folyamatunk virtuális memóriájába az mmap() hívással
 - Direktbe írni a regisztereket...
 - Milyen virtuális címre írunk, és milyen fizikaira (jó demó a virtuális memória kezelésre is)
 - Rendszergazda jogosultság kell hozzá
 - A /dev/mem megnyitása miatt
 - Mi lenne, ha ezt bárki megnyithatná olvasásra?